HashiCorp Vault — Day 3: PKI, Transit, Enterprise & Operations

Study material · Oct 16, 2026 · @Vivek Arora

Day 3 at a glance

Day 3 turns Vault into infrastructure: it issues certificates, encrypts application data without handing out keys, unseals itself, survives node loss, and is run, watched, automated and deployed on Kubernetes like any other production platform.

Learning objectives

1. Build a two-tier PKI (offline-style root, online intermediate) and issue, revoke and tidy certificates.
2. Use Transit for encryption as a service: encrypt, decrypt, rotate, rewrap, data keys, signing and HMAC.
3. Configure auto-unseal with a cloud KMS or the Transit seal, migrate from Shamir, and explain seal wrap.
4. Run Vault Agent with auto-auth, caching and templates — including certificates rendered with `pkiCert`.
5. Build a three-node Raft cluster, fail over the leader, and back up and restore with snapshots; explain performance and DR replication.
6. Expose telemetry to Prometheus, analyse audit logs, and apply production hardening practices.
7. Troubleshoot Vault from its logs, `vault debug` bundles and a repeatable decision tree.
8. Automate configuration and operations with Terraform and Ansible.
9. Implement Vault on Kubernetes: HA Helm deployment, TLS, auto-unseal, and the injector, CSI and Vault Secrets Operator patterns.
10. Combine PKI, Transit and Vault Agent in one end-to-end service.

Suggested agenda

#	Module	Duration	Labs
1	PKI secrets engine	60 min	Lab 3.1
2	Transit: encryption as a service	40 min	Lab 3.2

#	Module	Duration	Labs
3	Auto-unseal and seal wrap	35 min	Lab 3.3
4	Vault Agent and templating	40 min	Lab 3.4
5	HA, replication and disaster recovery	50 min	Lab 3.5
6	Monitoring, audit and best practices	35 min	Lab 3.6
7	Basic troubleshooting	30 min	Lab 3.7
8	Automation with Terraform and Ansible	50 min	Lab 3.8
9	Implementing Vault on Kubernetes	50 min	Lab 3.9
10	Capstone: PKI + Transit + Vault Agent	45 min	Lab 3.10

About 7 hours 15 minutes of content. For a shorter day, run Labs 3.8 and 3.9 as instructor demos.

Where Day 3 builds on Days 1 and 2

Day 3 topic	Builds on
PKI and Transit	KV and dynamic secrets: the same mount, role and lease model (Day 1 Module 6, Day 2 Module 1)
Auto-unseal	Shamir seal/unseal and the key hierarchy (Day 1 Module 2)
Vault Agent templating	AppRole and the Agent introduction (Day 2 Module 3, Lab 2.4)
HA and snapshots	Raft storage and the production server (Day 1 Module 3, Lab 1.3)
Monitoring and audit	Audit devices enabled in Labs 1.3 and 1.9
Vault on Kubernetes	Kubernetes auth and the injector (Day 2 Module 3, Lab 2.5)

Prerequisites and lab guide

Hands-on: every lab step, with expected-output screenshots, lives in the Vault Lab Guide — Day 3. Each module ends with the lab that practises it.

- The Day 1–2 lab machine: Vault 2.x, Docker, `jq`, `curl`, `openssl`, Python 3 with `hvac`.

- Day 3 additions: `nginx` (capstone), Terraform 1.6 or later, Ansible with the `community.hashi_vault` collection, and `kind`, `kubectl`, `helm` for Module 9.
- Optional: an AWS or Azure sandbox for cloud KMS auto-unseal; a Vault Enterprise trial licence to try replication and seal wrap.

Examples use Vault 2.1. Everything outside the Enterprise-only features works on Vault Community. All passwords, tokens and keys shown are lab-only examples.

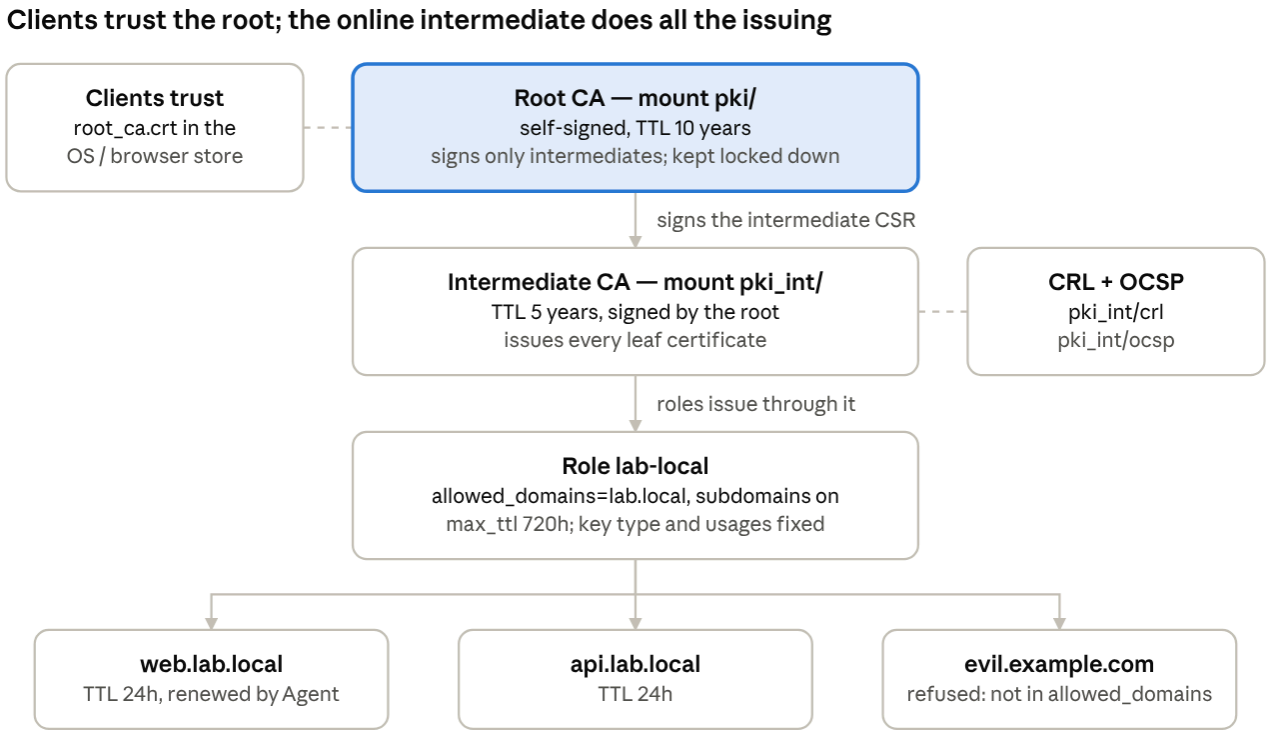
Module 1 — PKI secrets engine

Vault's PKI engine turns certificate issuance into an API call: no CSR emails, no spreadsheets of expiry dates, and certificates short-lived enough that most never need revoking.

Why use Vault as a CA

Traditional internal CA	Vault PKI
Tickets and manual CSRs; days to get a certificate	One authenticated API call; milliseconds
1–2 year certificates, outages when they expire	Hours or days; renewed automatically by Agent or cert-manager
Revocation lists that nobody checks	Short TTLs make revocation the exception; CRL and OCSP still available
Who requested what is hard to prove	Every issuance is in the audit log, tied to an identity and a policy

The two-tier hierarchy



Two-tier CA hierarchy · root, intermediate, role, leaves

The root signs one thing — the intermediate — and is otherwise idle; if the intermediate is ever compromised you revoke it and issue a new one without redistributing trust to every client.

In production the root often lives outside Vault (an offline or corporate CA) and only signs Vault's intermediate CSR; Lab 3.1 keeps both in Vault for simplicity.

Concepts

Concept	Meaning
Issuer	A CA certificate + key inside a PKI mount; a mount can hold several (for rotation), one is the default
Role	Rules for leaf certificates: allowed domains, SANs, TTLs, key type, usages
`issue` vs `sign`	`issue` makes the key pair inside Vault and returns it; `sign` signs a CSR whose key never leaves the client
CRL / OCSP	Revocation list and online status endpoints, published under the mount

Concept	Meaning
Tidy / auto-tidy	Removes expired certificates and revocation entries from storage
ACME	Lets ACME clients (certbot, Caddy, cert-manager) request certificates from Vault directly

Key commands (reference)

```
# Root CA
$ vault secrets enable pki
$ vault secrets tune -max-lease-ttl=87600h pki
$ vault write -field=certificate pki/root/generate/internal \
    common_name="StackMaster Root CA" issuer_name="root-2026" ttl=87600h >
root_ca.crt
$ vault write pki/config/urls \
    issuing_certificates="$VAULT_ADDR/v1/pki/ca"
crl_distribution_points="$VAULT_ADDR/v1/pki/crl"

# Intermediate CA: generate a CSR, have the root sign it, import the result
$ vault secrets enable -path=pki_int pki
$ vault secrets tune -max-lease-ttl=43800h pki_int
$ vault write -format=json pki_int/intermediate/generate/internal \
    common_name="StackMaster Intermediate CA" issuer_name="stackmaster-int" |
jq -r .data.csr > pki_int.csr
$ vault write -format=json pki/root/sign-intermediate issuer_ref="root-2026"
\
    csr=@pki_int.csr format=pem_bundle ttl=43800h | jq -r .data.certificate >
intermediate.pem
$ vault write pki_int/intermediate/set-signed certificate=@intermediate.pem

# Role and issuance
$ vault write pki_int/roles/lab-local allowed_domains="lab.local"
allow_subdomains=true max_ttl=720h
$ vault write pki_int/issue/lab-local common_name="web.lab.local" ttl=24h

# Revocation and housekeeping
$ vault write pki_int/revoke serial_number=<serial>
$ vault write pki_int/tidy tidy_cert_store=true tidy_revoked_certs=true
safety_buffer=72h
$ vault write pki_int/config/auto-tidy enabled=true interval_duration=12h
tidy_cert_store=true
$ vault pki health-check pki_int
```

Policy for an application that may only request its own certificates:

```
path "pki_int/issue/lab-local" {
  capabilities = ["update"]
}
```

Best practices

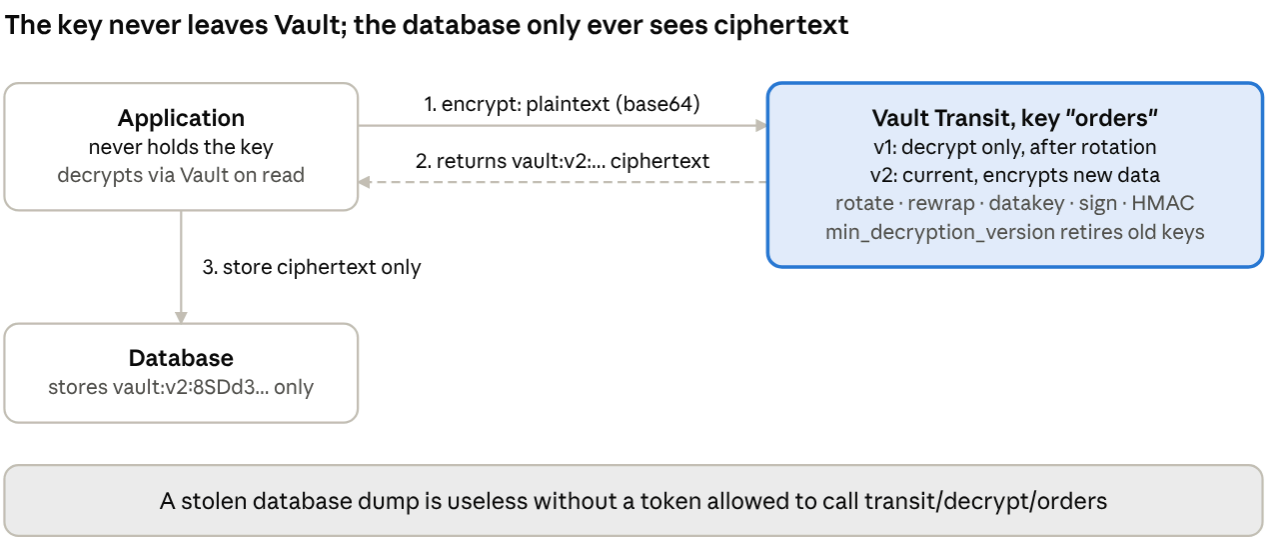
- Keep the root separate (another mount, another cluster, or offline) and use it only to sign intermediates.
- Prefer short leaf TTLs (hours to days) with automated renewal over long TTLs plus revocation.
- Use `no_store=true` on high-volume roles so short-lived certificates are not written to storage, and enable auto-tidy.
- Use one role per application or domain, with `allowed_domains` as narrow as possible and `allow_any_name=false`.
- Rotate issuers with multi-issuer support: import or generate a new intermediate, make it the default, let old certificates expire.
- For Kubernetes workloads, pair Vault PKI with cert-manager's Vault issuer or the Vault Secrets Operator `VaultPKISecret` (Module 9).

Hands-on

> **Do Lab 3.1 — Build a two-tier PKI and issue certificates** (35 min). Root and intermediate CA, a role, certificates verified with `openssl`, a refused domain, a revocation and the CRL, tidy and a health check, and signing a CSR (the HTTPS server comes in Lab 3.10).

Module 2 — Transit: encryption as a service

Transit gives applications cryptography without giving them keys: they send data to Vault and get ciphertext back, so key storage, rotation and access control stay in one audited place.



Transit encryption as a service · 3 steps

Transit stores nothing about the data it encrypts. Your database keeps the ciphertext; Vault keeps the keys and the policy that decides who may decrypt.

Operations

Operation	Path	Use
Encrypt / decrypt	`transit/encrypt/<key>`, `transit/decrypt/<key>`	Protect fields such as card numbers, Aadhaar, PAN
Rotate	`transit/keys/<key>/rotate`	New key version; old versions still decrypt
Rewrap	`transit/rewrap/<key>`	Re-encrypt old ciphertext with the newest version, without ever seeing the plaintext
Data key	`transit/datakey/plaintext/<key>`	Envelope encryption: encrypt large files locally with a one-off key that Vault wraps
Sign / verify	`transit/sign/<key>`, `transit/verify/<key>`	Signatures with ed25519, ECDSA or RSA keys
HMAC	`transit/hmac/<key>`	Tamper-evident hashes and keyed lookups

Ciphertext looks like `vault:v2:8SDd3WHDOjf7...`: the prefix records which key version encrypted it, which is what makes rotation painless.

Key types and options

Setting	Notes
`type=aes256-gcm96`	Default symmetric key; also `chacha20-poly1305`, `aes128-gcm96`
`type=ed25519`, `ecdsa-p256`, `rsa-2048` / `4096`	Asymmetric keys for signing (RSA also encrypts)
`exportable=false`	Default: the key can never be read out of Vault. Leave it that way
`auto_rotate_period=720h`	Vault rotates the key on a schedule
`min_decryption_version`	Oldest key version still allowed to decrypt — raise it after rewrapping
`derived=true`, `convergent_encryption=true`	Same plaintext + context gives the same ciphertext, so encrypted columns stay searchable

Key commands (reference)

```
$ vault secrets enable transit
$ vault write -f transit/keys/orders
$ vault write transit/encrypt/orders plaintext=$(echo -n "4111 1111 1111
1111" | base64)
$ vault write -field=plaintext transit/decrypt/orders
ciphertext="vault:v1:..." | base64 -d

$ vault write -f transit/keys/orders/rotate
$ vault write transit/rewrap/orders ciphertext="vault:v1:..."        # ->
vault:v2:...
$ vault write transit/keys/orders/config min_decryption_version=2

$ vault write -f transit/datakey/plaintext/orders                    #
envelope encryption
$ vault write -f transit/keys/signer type=ed25519
$ vault write transit/sign/signer input=$(echo -n "invoice-42" | base64)
```

Policies split encrypt and decrypt so a write-only service cannot read data back:

```
# orders-writer.hcl
path "transit/encrypt/orders" {
  capabilities = ["update"]
}

# orders-reader.hcl
path "transit/decrypt/orders" {
  capabilities = ["update"]
}
```

```
# transit from Python (hvac)
import base64, os, hvac

client = hvac.Client(url=os.environ["VAULT_ADDR"],
token=os.environ["VAULT_TOKEN"])
b64 = base64.b64encode(b"4111 1111 1111 1111").decode()
ct = client.secrets.transit.encrypt_data(name="orders", plaintext=b64)
["data"]["ciphertext"]
pt = client.secrets.transit.decrypt_data(name="orders", ciphertext=ct)
["data"]["plaintext"]
print(ct, base64.b64decode(pt).decode())
```

Vault Enterprise adds the Transform engine (format-preserving encryption, masking, tokenization) for data that must keep its shape, such as card numbers that still look like card numbers.

Best practices

- One key per data domain (orders, customers, payroll), so access and rotation are independent.
- Turn on `auto_rotate_period` and schedule a rewrap job; then raise `min_decryption_version`.
- Send large payloads through data keys (envelope encryption), not through `encrypt`.
- Use `batch_input` to encrypt or decrypt many values in one request.
- Never mark keys `exportable` or allow `plaintext` backups unless a documented migration needs it.

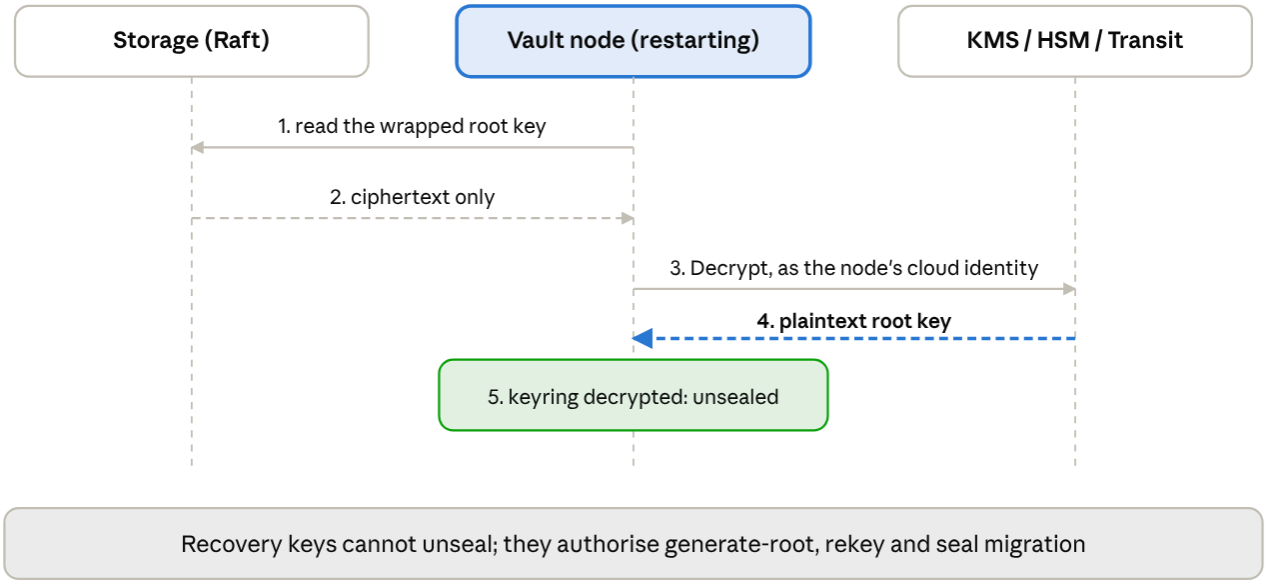
Hands-on

> **Do Lab 3.2 — Transit encryption as a service** (25 min). Encrypt and decrypt, rotate and rewrap, retire old versions, generate a data key, sign and verify, and give an app an encrypt/decrypt-only policy.

Module 3 — Auto-unseal and seal wrap

Auto-unseal moves protection of the root key from people holding Shamir shares to a key in a KMS or HSM, so a restarted or rescheduled Vault node unseals itself in seconds — essential for clusters, Kubernetes and autoscaling.

With auto-unseal a restart needs no key holders: the KMS unwraps the root key



Auto-unseal on restart · 5 steps, no key holders

The trust moves to the KMS: whoever can call `Decrypt` on that key with the node's identity can unseal Vault, so the KMS key policy becomes part of Vault's security boundary.

Supported seals

Seal stanza	Key lives in	Node authenticates with
`awskms`	AWS KMS	EC2/EKS IAM role (or access keys)
`azurekeyvault`	Azure Key Vault	Managed identity (or service principal)
`gcpckms`	Google Cloud KMS	Service account
`transit`	Another Vault's Transit engine	A Vault token for that cluster
`pkcs11` (Enterprise)	Hardware security module	HSM PIN / slot
`ocikms`, `alicloudkms`	Oracle / Alibaba Cloud KMS	Instance principal / RAM role

Configuration examples

```
# AWS KMS — the node's IAM role needs kms:Encrypt, kms:Decrypt,
kms:DescribeKey on this key
seal "awskms" {
  region     = "ap-south-1"
  kms_key_id = "alias/vault-unseal"
}
```

```
# Azure Key Vault — with a managed identity, tenant/client settings can be
omitted;
# the identity needs the "Key Vault Crypto User" role (wrapKey/unwrapKey)
seal "azurekeyvault" {
  vault_name = "kv-vault-unseal"
  key_name   = "vault-unseal-key"
}
```

```
# Transit seal — a small, separately managed "unsealer" Vault holds the key
(Lab 3.3)
seal "transit" {
  address         = "http://unsealer.lab.local:8200"
  token           = "<periodic token limited to
transit/encrypt|decrypt/autounseal>"
  key_name        = "autounseal"
  mount_path      = "transit/"
  disable_renewal = "false"
}
```

Initialise an auto-unsealed cluster with recovery keys instead of unseal keys:

```
$ vault operator init -recovery-shares=5 -recovery-threshold=3
```

Seal migration

Direction	Procedure (single node; repeat per node in a cluster, standbys first)
Shamir → auto-unseal	Add the `seal` stanza, restart, then `vault operator unseal -migrate` with the Shamir unseal-key threshold. The old unseal keys become recovery keys
Auto-unseal → Shamir	Add `disabled = "true"` to the seal stanza, restart, then `vault operator unseal -migrate` with the recovery-key threshold
KMS → another KMS	Keep the old stanza with `disabled = "true"`, add the new one, restart, `vault operator unseal -migrate` with recovery keys

Take a Raft snapshot before any migration, and keep the old KMS key until every node has migrated.

Seal wrap (Enterprise)

Seal wrap encrypts selected values a second time with the seal's KMS or HSM key, on top of Vault's barrier encryption — the basis of Vault Enterprise's FIPS 140 compliance story. Critical values (the root key, the keyring, recovery keys) are seal-wrapped automatically when a supporting seal is configured, and mounts can opt in:

```
$ vault secrets enable -seal-wrap -path=pci-transit transit      # Enterprise
only
```

Best practices

- Give the KMS key its own tight key policy; only Vault node identities may use it, and deny key deletion.
- Store recovery keys like Shamir keys: split among several people, ideally PGP-encrypted.
- Monitor KMS availability: a KMS outage does not seal a running Vault, but nodes cannot unseal after a restart until it returns.
- For the Transit seal, the unsealer cluster must be more available than the cluster it unseals; keep it small, separate and simple.

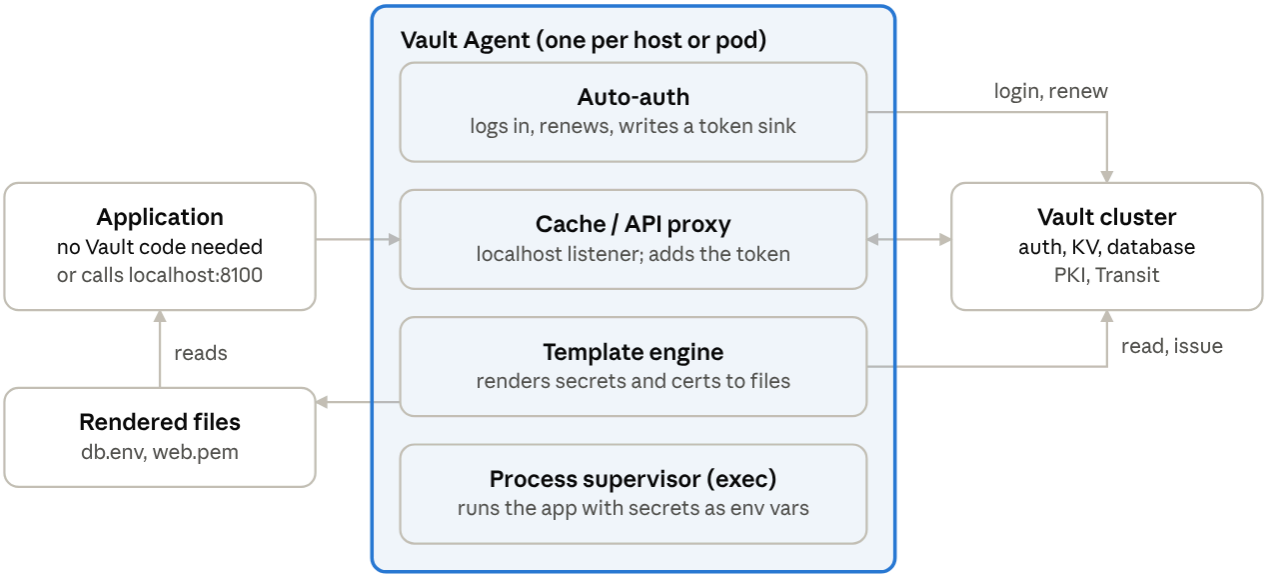
Hands-on

> **Do Lab 3.3 — Auto-unseal with the Transit seal** (25 min). One Vault acts as the unsealer; a second Raft-backed Vault initialises with recovery keys and unseals itself after every restart.

Module 4 — Vault Agent and templating

Vault Agent is a client-side daemon that logs in, keeps tokens and leases alive, and turns secrets into files or environment variables, so applications can use Vault without a single line of Vault code. Day 2 (Lab 2.4) introduced it; this module covers the full feature set.

Agent does the Vault work so the application only reads a file



Vault Agent components · 4 jobs, one sidecar or daemon

Each box is optional: use auto-auth plus templates for most apps, add the cache/proxy for apps that call the Vault API themselves, and use process-supervisor mode when the app reads configuration only from environment variables.

Block	Purpose
`auto_auth`	Logs in with AppRole, Kubernetes, AWS, Azure, JWT, cert and more; re-authenticates automatically
`sink`	Writes the token (optionally response-wrapped) to a file for other processes
`cache` + `api_proxy` + `listener`	Local endpoint that injects the auto-auth token and caches leases (Vault Proxy is the dedicated binary for this role)
`template` / `template_config`	Renders Consul-Template files; re-renders before leases or certificates expire
`env_template` + `exec`	Process-supervisor mode: starts the app with secrets as environment variables and restarts it when they change

A complete agent configuration

```
# /etc/vault-agent/agent.hcl
pid_file = "/run/vault-agent.pid"

vault {
  address = "http://127.0.0.1:8200"
}

auto_auth {
  method "approle" {
    config = {
      role_id_file_path                  = "/etc/vault-agent/role_id"
      secret_id_file_path                = "/etc/vault-agent/secret_id"
      remove_secret_id_file_after_reading = false
    }
  }
  sink "file" {
    config = { path = "/run/vault-agent/token" }
  }
}

cache {}

api_proxy {
  use_auto_auth_token = true
}

listener "tcp" {
  address     = "127.0.0.1:8100"
  tls_disable = true
}

template_config {
  static_secret_render_interval = "5m"     # how often KV secrets are re-read
  exit_on_retry_failure         = true     # fail loudly instead of serving
stale config
}

template {
  source      = "/etc/vault-agent/app.ctmpl"
  destination = "/etc/app/app.env"
  perms       = "0640"
}

template {
```

```
  source      = "/etc/vault-agent/web.ctmpl"
  destination = "/etc/nginx/certs/web.pem"
  perms       = "0640"
  exec {
    command = ["systemctl", "reload", "nginx"]   # only when the rendered
file changes
  }
}
```

Templates

```
{{/* app.ctmpl — KV v2 values and a dynamic database credential */}}
{{ with secret "secret/data/payments/config" -}}
GATEWAY_URL={{ .Data.data.gateway_url }}
{{ end -}}
{{ with secret "database/creds/payments-ro" -}}
DB_USER={{ .Data.username }}
DB_PASS={{ .Data.password }}
{{ end -}}
```

```
{{/* web.ctmpl — a certificate from Vault PKI; key written to its own file
*/}}
{{ with pkiCert "pki_int/issue/lab-local" "common_name=web.lab.local"
"ttl=24h" -}}
{{ .Cert }}{{ .CA }}
{{ .Key | writeToFile "/etc/nginx/certs/web.key" "root" "root" "0600" }}
{{- end }}
```

`pkiCert` uses the destination file as its cache: it requests a new certificate only when the one in the file is close to expiry, so the file must contain the certificate itself.

Function	Use
`secret "path" "k=v"`	Read (or write, for parameters) any Vault path
`pkiCert "pki/issue/role" "common_name=..."`	Certificate with `.Cert`, `.Key`, `.CA`, renewed before expiry
`writeToFile path user group mode`	Write a value (such as a private key) to a second file
`toJSON`, `toYAML`, `base64Decode`, `env`	Format conversion and environment lookups
`with`, `range`, `if`	Go template control flow; `-` trims whitespace

Process-supervisor mode

```
# For apps that only read environment variables
env_template "DB_PASS" {
  contents             = "{{ with secret \"database/creds/payments-ro\" }}{{
.Data.password }}{{ end }}"
  error_on_missing_key = true
}

exec {
  command                   = ["python3", "/opt/app/app.py"]
  restart_on_secret_changes = "always"
  restart_stop_signal       = "SIGTERM"
}
```

Running Agent as a service

```
# /etc/systemd/system/vault-agent.service
[Unit]
Description=Vault Agent
After=network-online.target

[Service]
ExecStart=/usr/bin/vault agent -config=/etc/vault-agent/agent.hcl
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

Best practices

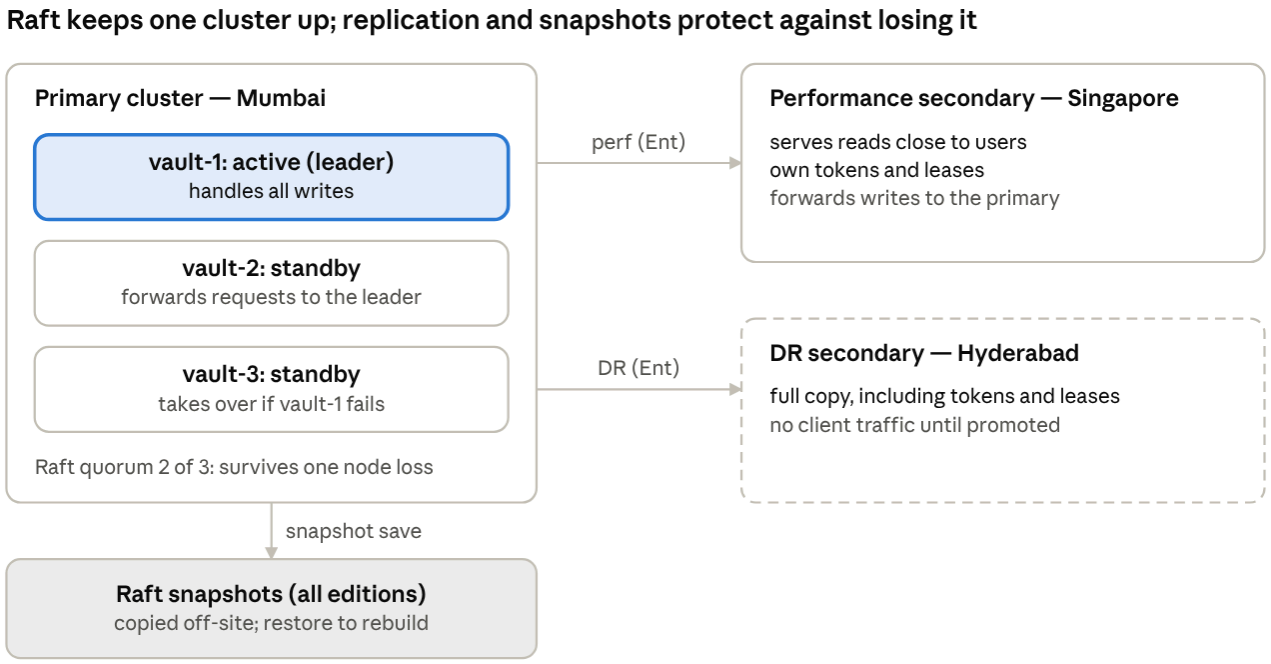
- One Agent per application (or per pod) with its own AppRole or platform identity — never share a token between apps.
- Prefer `exec` reload commands over cron-based restarts; they run only when the rendered output changes.
- Set `exit_on_retry_failure = true` under systemd or Kubernetes so a broken Agent is restarted rather than silently stale.
- Keep rendered files on tmpfs (`/run`) where possible, with tight `perms`.

Hands-on

> **Do Lab 3.4 — Vault Agent templating, caching and certificate rendering** (35 min). Render KV and PKI templates, run a command on change, use the local API proxy, and run an app in process-supervisor mode.

Module 5 — High availability, replication and disaster recovery

A three-node Raft cluster survives the loss of any one node with automatic leader election; replication (Enterprise) protects against losing a whole cluster or region; snapshots protect against everything else, including mistakes.



HA, replication and backup topology · 3 clusters

HA keeps you running through a node failure; DR gets you back after a site failure; snapshots get you back after an operator error that replication would faithfully copy everywhere.

High availability with Integrated Storage (Raft)

Cluster size	Quorum	Node failures tolerated
1	1	0
3	2	1
5	3	2

- One node is **active** (the Raft leader); the others are **standbys** that forward client requests to it. Vault Enterprise adds **performance standbys** that serve reads locally.
- Use an odd number of voters (3 or 5) spread across failure zones; more voters slow writes without adding safety.
- **Autopilot** watches server health, removes dead servers (when configured), and waits for new servers to stabilise before they vote.

```
# vault.hcl on each node — nodes find each other with retry_join
storage "raft" {
  path    = "/opt/vault/data"
  node_id = "vault-2"
  retry_join {
    leader_api_addr = "https://vault-1.lab.local:8200"
  }
  retry_join {
    leader_api_addr = "https://vault-3.lab.local:8200"
  }
}
```

```
$ vault operator raft list-peers                  # who is leader, who votes
$ vault operator raft join http://vault-1:8200     # manual join (if not
using retry_join)
$ vault operator raft remove-peer vault-3         # remove a dead node
$ vault operator raft autopilot state             # health, failure tolerance
$ vault operator raft autopilot set-config -cleanup-dead-servers=true \
    -dead-server-last-contact-threshold=10m -min-quorum=3
$ vault operator step-down                        # force a leader election
(maintenance)
```

Snapshots: backup and restore

```
$ vault operator raft snapshot save /backup/vault-$(date +%F-%H%M).snap
$ vault operator raft snapshot restore /backup/vault-2026-10-16-0200.snap
$ vault operator raft snapshot restore -force /backup/other-cluster.snap   #
from a different cluster
```

A snapshot contains all encrypted data, so restoring it needs the same unseal or KMS keys. Vault Enterprise can schedule snapshots to S3, Azure Blob or GCS (`sys/storage/raft/snapshot-auto/config`); on Community, schedule `snapshot save` with cron, Ansible (Module 8) or a Kubernetes CronJob.

Replication (Enterprise)

	Performance replication	Disaster-recovery replication
Purpose	Scale reads, serve other regions	Warm standby for site failure
What replicates	Configuration, policies, secrets engines, KV data	Everything, including tokens and leases
Clients	Served by the secondary; writes forwarded to the primary	None until promoted
Tokens	Secondary issues its own	Same tokens keep working after promotion
Failover	Not a failover target on its own	`sys/replication/dr/secondary/promote` with a DR operation token

Paths can be filtered from performance secondaries (for example to keep EU data in the EU) with paths filters.

DR runbook

Scenario	Recovery
One node lost (3-node cluster)	Nothing to do immediately; replace the node, let it `retry_join`, and autopilot removes the dead peer
Quorum lost (2 of 3 nodes down)	Recover a node, or on a surviving node write `raft/peers.json` listing the remaining servers and restart to force a new quorum

Scenario	Recovery
Whole cluster or region lost	Enterprise: promote the DR secondary and repoint DNS. Community: build a new cluster and restore the latest snapshot
Accidental deletion or bad config	Restore a snapshot to a separate cluster, extract what is needed, or restore over production during a maintenance window
KMS key lost or deleted	Unrecoverable without the key: protect KMS keys from deletion and back them up per the cloud's guidance

Test restores regularly. A backup that has never been restored is a hope, not a backup.

Hands-on

> **Do Lab 3.5 — Three-node Raft cluster, failover, snapshots** (35 min). Three Vault nodes on one machine, leader failover by stopping the active node, autopilot state, and a snapshot save and restore that brings back a deleted secret.

Module 6 — Monitoring, audit devices and best practices

Vault in production needs three kinds of visibility. **Health** tells you whether each node is up, unsealed and in the cluster. **Telemetry** (metrics) tells you how it is performing and whether it is heading for trouble. **Audit logs** tell you who did what. Day 1 covered `vault status`; this module adds the other two and ends with the hardening checklist.

6.1 The health endpoint

`GET /v1/sys/health` needs no token, so it is what load balancers and Kubernetes probes call. The HTTP status code carries the answer:

Code	Meaning	Typical use
200	Initialized, unsealed, active	LB sends writes here
429	Unsealed standby	Healthy, but not the leader
472	DR secondary (Enterprise)	Do not route clients
473	Performance standby (Enterprise)	Can serve reads

Code	Meaning	Typical use
501	Not initialized	Run `vault operator init`
503	Sealed	Unseal or check the KMS

Query parameters change the codes: `?standbyok=true` makes a standby return 200 (useful for a readiness probe), and `?sealedcode=200&uninitcode=200` turns a probe into a liveness check that only fails when the process is dead.

```
curl -s -o /dev/null -w '%{http_code}\n' $VAULT_ADDR/v1/sys/health
curl -s $VAULT_ADDR/v1/sys/health | jq '{initialized, sealed, standby,
version, cluster_name}'
```

6.2 Telemetry

Vault keeps metrics in memory and exposes them through `/v1/sys/metrics`. Enable the Prometheus format by setting a retention time in the server config:

```
telemetry {
  prometheus_retention_time = "24h"   # 0 disables the Prometheus endpoint
  disable_hostname          = true    # metric names without the host prefix
  usage_gauge_period        = "10m"   # how often counts like token/entity
totals are taken
}
```

The endpoint needs a token with `read` on `sys/metrics` (or `unauthenticated_metrics_access = true` in the listener's own `telemetry` block). A Prometheus scrape job:

```
scrape_configs:
  - job_name: vault
    metrics_path: /v1/sys/metrics
    params:
      format: [prometheus]
    scheme: https
    tls_config:
      ca_file: /etc/prometheus/vault-ca.pem
    authorization:
      credentials_file: /etc/prometheus/vault-token   # token from a metrics-
only policy
    static_configs:
      - targets: ["vault-1:8200", "vault-2:8200", "vault-3:8200"]
```

```
# policy: prometheus-metrics
path "sys/metrics" {
  capabilities = ["read"]
}
```

Metrics worth an alert (Prometheus names; dots become underscores):

Metric	Watch for	Why it matters
`vault_core_unsealed`	0 on any node	Node sealed — KMS or restart issue
`vault_core_active`	no node reporting 1	No leader — cluster cannot write
`vault_raft_leader_lastContact`	p99 > 500 ms	Followers losing the leader; network or disk
`vault_autopilot_healthy`	0	A Raft peer is unhealthy
`vault_expire_num_leases`	steady climb	Lease explosion — TTLs too long or apps not reusing creds
`vault_token_count`	steady climb	Apps logging in on every request
`vault_audit_log_request_failure`	> 0	Audit device failing — Vault may block requests
`vault_core_handle_request`	p99 latency rising	Overloaded node or slow storage
`vault_runtime_alloc_bytes`	near the memory limit	Risk of OOM kill

For a quick look without Prometheus, `vault read sys/metrics` prints the same data as JSON, and `vault operator raft autopilot state` shows Raft health per node.

6.3 Audit devices

An audit device records every request and response that passes through the Vault API. It is the evidence trail for compliance (PCI DSS, SOC 2, ISO 27001), and it is the first place to look when someone asks "who read this secret?".

Type	Writes to	Notes
`file`	A local file (or `stdout`)	Most common; ship with Fluent Bit, Filebeat or the Splunk forwarder

Type	Writes to	Notes
`syslog`	Local syslog daemon	Not on Windows; message size limits apply
`socket`	TCP/UDP endpoint	Straight to a log collector; TCP is safer than UDP

```
vault audit enable file file_path=/var/log/vault/vault-audit.log
vault audit enable -path=syslog_audit syslog tag=vault facility=AUTH
vault audit list -detailed
```

Three behaviours to know:

- **Sensitive values are HMAC'd.** Tokens, passwords and secret values appear as `hmac-sha256:…`, never in clear text. To check whether a known value appears in the log, hash it with the same device: `vault write sys/audit-hash/file input="s3cr3t"`, then search the log for the result.
- **Vault fails closed.** If it cannot write to *any* enabled audit device, it refuses the request. Enable at least two devices (for example `file` and `socket`) so one full disk does not stop the service.
- **Log rotation needs a signal.** After `logrotate` moves the file, send `SIGHUP` to Vault so it reopens the file (`postrotate systemctl reload vault`).

Each line is one JSON object, `type` `request` or `response`. Useful `jq` queries:

```
# Who touched secret/data/webapp in the last log file?
jq -c 'select(.type=="response" and .request.path=="secret/data/webapp")
       | {time, user: .auth.display_name, op: .request.operation, ip:
.request.remote_address}' \
   /var/log/vault/vault-audit.log

# Permission-denied errors, grouped by path
jq -r 'select(.error != null and (.error|test("permission denied"))) |
.request.path' \
   /var/log/vault/vault-audit.log | sort | uniq -c | sort -rn
```

6.4 Production hardening checklist

HashiCorp's production hardening guide, condensed into what to check before go-live:

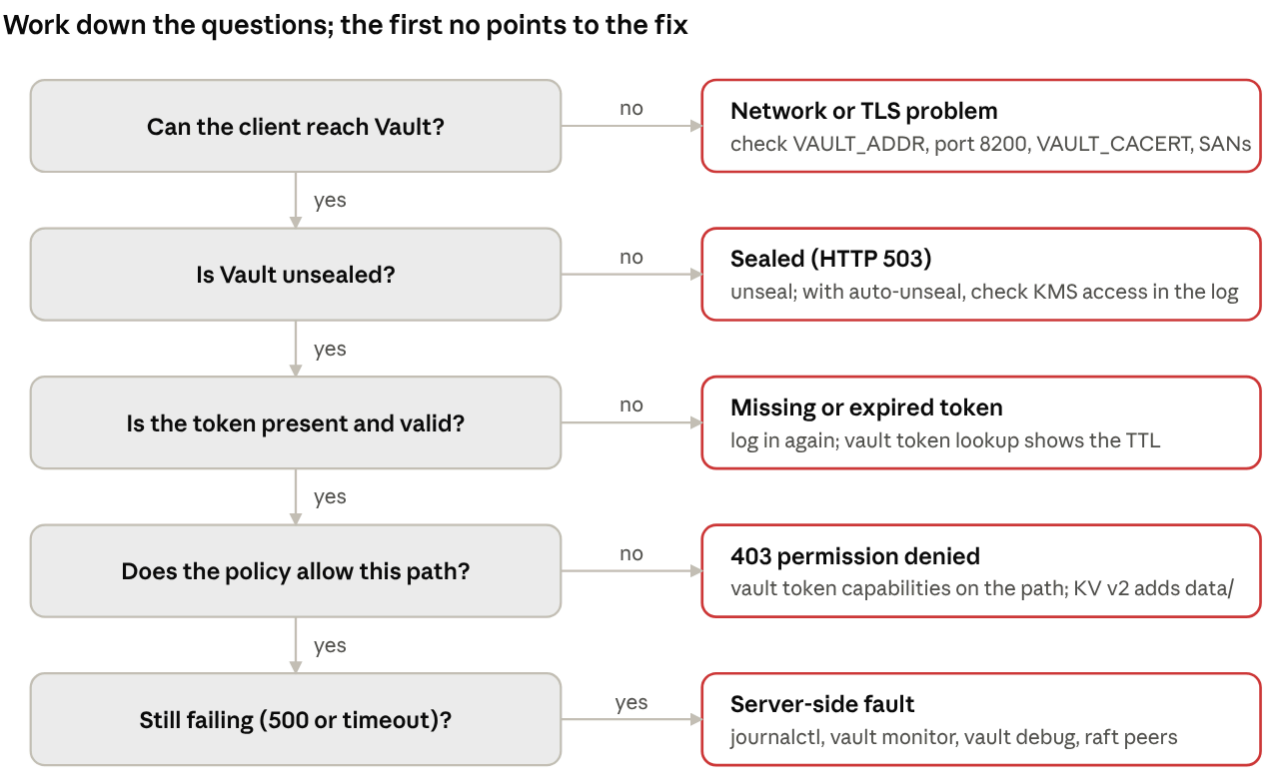
- [ ] **TLS end to end** — `tls_disable = false`; clients verify the CA; TLS 1.2 minimum.
- [ ] **Revoke the initial root token** after setup; use `vault operator generate-root` with key or recovery-key holders when one is genuinely needed.
- [ ] **Run as a dedicated non-root user** with systemd hardening (`ProtectSystem=full`, `NoNewPrivileges=yes`); disable swap or keep `mlock` working; disable core dumps.

- [ ] **Auto-unseal** with a cloud KMS or HSM (Module 3); store recovery keys with separate custodians.
- [ ] **At least two audit devices**, shipped off the host to a SIEM.
- [ ] **Least-privilege policies** (Day 2 Module 2) — no human uses `root`; break-glass access is audited.
- [ ] **Short TTLs** on tokens and leases; prefer dynamic secrets (Day 2 Module 1).
- [ ] **Raft cluster of 3 or 5 nodes** across failure zones, with automated snapshots copied off-site (Module 5).
- [ ] **Firewall** — 8200 (API) open to clients only; 8201 (cluster) between Vault nodes only.
- [ ] **Telemetry and alerts** on the metrics in 6.2; health probes on every node.
- [ ] **Upgrade plan** — follow the release notes; upgrade standbys first, then step down the leader.
- [ ] **Restrict `sys/`** — only operators get `sys/*`; apps never do.

> **Do Lab 3.6 — Telemetry, Prometheus and audit devices** in the Day 3 Lab Guide: enable Prometheus metrics, scrape them with a metrics-only token, enable two audit devices, trace a secret read through the audit log with `jq`, and confirm HMAC with `sys/audit-hash`.

Module 7 — Basic troubleshooting: logs and common issues

Most Vault problems fall into a handful of layers: the client cannot reach the server, the server is sealed, the token is wrong, the policy is wrong, or something is broken on the server side. Work through them in that order — each check takes seconds and rules out a whole layer.



troubleshooting decision tree · 5 checks, top to bottom

The error text usually names the layer. Read the whole message: Vault prints the HTTP code, the URL and the reason (`Code: 403. Errors: * permission denied`).

7.1 Where the logs are

Where Vault runs	Read the server log with
systemd (package install)	`journalctl -u vault -f` · `journalctl -u vault --since "10 min ago" -p warning`
Docker	`docker logs -f vault`
Kubernetes	`kubectl logs vault-0 -c vault -n vault` · `kubectl describe pod vault-0 -n vault` for events
Log file in config	`log_file = "/var/log/vault/vault.log"` with `log_rotate_max_files`
Any of the above, over the API	`vault monitor -log-level=debug` (needs `sudo` and `read` on `sys/monitor`)

Raise detail in the server config with `log_level = "debug"` (values `trace`, `debug`, `info`, `warn`, `error`) and `log_format = "json"` for log shippers. A `SIGHUP` (`systemctl reload vault`) reloads the log level without a restart. Do not leave `trace` on in production: it is very noisy.

The server log and the audit log answer different questions. The **server log** says what Vault itself did (unseal, leader election, storage errors, plugin crashes). The **audit log** (Module 6) says what clients asked for and what they got back, including every `permission denied`.

7.2 Diagnostic commands

```
vault status                                    # sealed? HA mode? active
node?
vault token lookup                              # whose token, policies, TTL
left
vault token capabilities secret/data/webapp     # what this token may do on a
path
vault read sys/seal-status                      # seal type, progress,
recovery seal
vault operator raft list-peers                  # leader, voters, followers
vault operator raft autopilot state             # Raft health per node
vault lease lookup <lease_id>                   # TTL left on a dynamic secret
vault kv get -output-curl-string secret/webapp # the exact API call the CLI
makes
vault debug -duration=2m -interval=30s -output=/tmp/vault-debug.tar.gz
```

`vault debug` collects metrics, goroutine and heap profiles, replication status, host information and the sanitised server config into one archive — the bundle to attach to a support ticket. It needs a token that can read `sys/pprof`, `sys/metrics`, `sys/host-info` and `sys/config/state/sanitized`.

7.3 Common issues and fixes

Symptom (error text)	Likely cause	Fix
`dial tcp 127.0.0.1:8200: connect: connection refused`	Server not running, or wrong `VAULT_ADDR`	`systemctl status vault`; check the listener address and `VAULT_ADDR`
`http: server gave HTTP response to HTTPS client`	`VAULT_ADDR` uses https but the listener has `tls_disable = true`	Make the scheme match the listener

Symptom (error text)	Likely cause	Fix
`x509: certificate signed by unknown authority`	Client does not trust the CA	`export VAULT_CACERT=/path/ca.pem` (never `VAULT_SKIP_VERIFY` in production)
`x509: certificate is valid for vault-1, not 10.0.0.5`	Address not in the cert's SANs	Reissue the cert with that DNS name or IP SAN
`Code: 503 ... Vault is sealed`	Restart without auto-unseal, or KMS unreachable	Unseal; with auto-unseal, read the server log for the KMS error
`AccessDeniedException` / `kms:Decrypt` in the log	Vault's IAM role lacks KMS rights	Grant `kms:Encrypt`, `kms:Decrypt`, `kms:DescribeKey` on the key
`Code: 400 ... missing client token`	`VAULT_TOKEN` not set, or Agent sink empty	`vault login`, or check the Agent's auto_auth log
`Code: 403 ... permission denied`	Policy does not cover the path, or token expired	`vault token capabilities <path>`; for KV v2 the policy path is `secret/data/...`; check the audit log
`Error initializing core: Failed to lock memory`	No `IPC_LOCK` capability	Add the capability, or set `disable_mlock = true` (recommended with Raft storage)
`no leader` / `failed to make requestVote RPC`	Port 8201 blocked or wrong `cluster_addr`	Open 8201 between nodes; `cluster_addr` must be reachable by the peers
`invalid lease ID` on renew	Lease expired, revoked or reached `max_ttl`	Request a new secret; review TTLs (Day 2 Module 4)
AppRole login `invalid secret id`	SecretID used up (`secret_id_num_uses`) or expired	Issue a fresh SecretID; deliver it with response wrapping (Day 2 Module 5)
Kubernetes login `permission denied`	ServiceAccount or namespace not in the role, or wrong audience	`vault read auth/kubernetes/role/<role>`; match `bound_service_account_*` and `audience`

Symptom (error text)	Likely cause	Fix
`context deadline exceeded`	Network latency or slow storage	Check disk latency and `vault_raft_leader_lastContact`; raise `VAULT_CLIENT_TIMEOUT` only as a stopgap

7.4 Recovering from a lost leader or quorum

If one of three Raft nodes fails, the cluster keeps working; replace it with `vault operator raft remove-peer` and join a new node. If two of three fail, there is no quorum and Vault cannot elect a leader. The documented recovery is to stop Vault on the surviving node, write a `raft/peers.json` file in its data directory listing only that node, and start it again — it comes up as a single-node cluster, and you join new nodes to it. Restoring the latest snapshot into a new cluster (Module 5) is the alternative.

> **Do Lab 3.7 — Break-fix troubleshooting** in the Day 3 Lab Guide: fix five deliberately broken setups (wrong scheme, untrusted CA, sealed node, KV v2 policy path, unwritable storage directory) using the logs and commands above, then collect a `vault debug` bundle.

Module 8 — Automation with Terraform and Ansible

Everything so far has been typed at a terminal. That is right for learning and wrong for production: hand-made changes are not reviewed, not repeatable and not recorded anywhere except the audit log. The two tools split the work naturally:

	Terraform (`hashicorp/vault` provider)	Ansible (`community.hashi_vault` collection)
Best at	Vault's **configuration** — mounts, policies, auth roles, PKI and Transit setup	**Operational procedures** — health checks, snapshots, rotations, issuing certs, pushing secrets to hosts
Style	Declarative; plan shows the diff; drift detection	Procedural playbooks; idempotent modules
Runs	In CI on every merge to the Vault config repo	On a schedule (AWX/AAP, cron) or on demand
Watch out	State file can hold secret values	Keep secret output out of logs with `no_log: true`

Both authenticate to Vault like any other client: give the pipeline its own AppRole (or JWT auth from the CI system, Day 2 Module 3) with a policy that covers only what it manages — never a root token.

8.1 Terraform: Vault configuration as code

```
# versions.tf
terraform {
  required_version = ">= 1.6"
  required_providers {
    vault = {
      source  = "hashicorp/vault"
      version = "~> 5.0"
    }
  }
  backend "s3" {               # remote, encrypted state — it may contain
secrets
    bucket  = "stackmaster-tfstate"
    key     = "vault/config.tfstate"
    region  = "ap-south-1"
    encrypt = true
  }
}

provider "vault" {
  # address and token come from VAULT_ADDR / VAULT_TOKEN, or log in with
AppRole:
  auth_login {
    path = "auth/approle/login"
    parameters = {
      role_id   = var.tf_role_id
      secret_id = var.tf_secret_id
    }
  }
}
```

The same objects you built by hand on Days 1–3, declared once:

```
# main.tf
resource "vault_mount" "kv" {
  path    = "secret"
  type    = "kv"
  options = { version = "2" }
}

resource "vault_policy" "webapp" {
```

```
  name   = "webapp"
  policy = file("${path.module}/policies/webapp.hcl")
}

resource "vault_auth_backend" "approle" {
  type = "approle"
}

resource "vault_approle_auth_backend_role" "webapp" {
  backend        = vault_auth_backend.approle.path
  role_name      = "webapp"
  token_policies = [vault_policy.webapp.name]
  token_ttl      = 3600
  token_max_ttl  = 14400
  secret_id_ttl  = 86400
}

# Transit — encryption as a service
resource "vault_mount" "transit" {
  path = "transit"
  type = "transit"
}

resource "vault_transit_secret_backend_key" "orders" {
  backend            = vault_mount.transit.path
  name               = "orders"
  type               = "aes256-gcm96"
  auto_rotate_period = 2592000   # 30 days, in seconds
  deletion_allowed   = false
}

# PKI — root CA and a role (intermediate pattern: see the lab)
resource "vault_mount" "pki" {
  path                  = "pki"
  type                  = "pki"
  max_lease_ttl_seconds = 315360000   # 10 years
}

resource "vault_pki_secret_backend_root_cert" "root" {
  backend     = vault_mount.pki.path
  type        = "internal"
  common_name = "StackMaster Root CA"
  ttl         = "87600h"
  issuer_name = "root-2026"
}
```

```
resource "vault_pki_secret_backend_role" "web" {
  backend          = vault_mount.pki.path
  name             = "web-server"
  allowed_domains  = ["stackmaster.lab"]
  allow_subdomains = true
  max_ttl          = "720h"
  key_type         = "rsa"
  key_bits         = 2048
}

resource "vault_audit" "file" {
  type    = "file"
  options = { file_path = "/var/log/vault/vault-audit.log" }
}
```

The workflow is the usual one — `terraform init`, `terraform plan` (review the diff in the pull request), `terraform apply`. Day-2 operations become code changes: onboarding an app is a new policy file plus a role; rotating a Transit key is not a Terraform change: `terraform apply -replace` would destroy the key and every ciphertext made with it, so call `transit/keys/<name>/rotate` instead (Ansible below).

Three rules for Terraform with Vault:

- **Treat the state file as a secret.** Anything Terraform reads or writes (a `vault_kv_secret_v2` value, a generated token) lands in state. Use an encrypted remote backend with tight access. With Terraform 1.11+ and provider 5.x, write-only arguments such as `data_json_wo` keep a value out of state.
- **Configure Vault with Terraform; do not deliver app secrets with it.** Apps should fetch their own secrets at run time (Agent, VSO — Modules 4 and 9).
- **Split state by blast radius** — for example one workspace for auth and policies, one for secrets engines.

8.2 Ansible: operational playbooks

```
ansible-galaxy collection install community.hashi_vault
pip install hvac                       # the Python client the collection
uses
export ANSIBLE_HASHI_VAULT_ADDR=https://vault.stackmaster.lab:8200
export ANSIBLE_HASHI_VAULT_CA_CERT=/etc/vault/ca.pem
```

Set the connection once with `module_defaults`, then each task only names what it does:

```
# vault-ops.yml — routine operations against Vault
- name: Vault operational tasks
  hosts: localhost
  gather_facts: true
  module_defaults:
    group/community.hashi_vault.vault:
      url: "{{ vault_addr }}"
      auth_method: approle
      role_id: "{{ ops_role_id }}"
      secret_id: "{{ ops_secret_id }}"

  tasks:
    - name: Health check (200 = active, 429 = standby)
      ansible.builtin.uri:
        url: "{{ vault_addr }}/v1/sys/health"
        status_code: [200, 429]
        ca_path: /etc/vault/ca.pem
      register: health

    - name: Log in once and reuse the token
      community.hashi_vault.vault_login:
      register: login

    - name: Take a Raft snapshot
      ansible.builtin.uri:
        url: "{{ vault_addr }}/v1/sys/storage/raft/snapshot"
        headers:
          X-Vault-Token: "{{ login.login.auth.client_token }}"
        dest: "/backup/vault-{{ ansible_date_time.iso8601_basic_short
}}.snap"
        ca_path: /etc/vault/ca.pem
      no_log: true

    - name: Rotate the Transit key for orders
      community.hashi_vault.vault_write:
        path: transit/keys/orders/rotate

    - name: Write a KV v2 secret
      community.hashi_vault.vault_kv2_write:
        engine_mount_point: secret
        path: webapp
        data:
          db_user: webapp
          db_password: "{{ new_db_password }}"
      no_log: true
```

Consuming secrets on managed hosts — a lookup reads the secret on the control node and the template writes it onto the host:

```
# deploy-webapp.yml
- name: Deploy webapp config with secrets from Vault
  hosts: web
  vars:
    db: "{{ lookup('community.hashi_vault.vault_kv2_get', 'webapp',
             engine_mount_point='secret', url=vault_addr,
             auth_method='approle', role_id=app_role_id,
secret_id=app_secret_id) }}"
  tasks:
    - name: Render application.properties
      ansible.builtin.template:
        src: application.properties.j2   # uses {{ db.secret.db_password }}
        dest: /opt/webapp/application.properties
        owner: webapp
        mode: "0600"
      no_log: true

    - name: Issue a TLS certificate from Vault PKI
      community.hashi_vault.vault_pki_generate_certificate:
        url: "{{ vault_addr }}"
        auth_method: approle
        role_id: "{{ app_role_id }}"
        secret_id: "{{ app_secret_id }}"
        engine_mount_point: pki_int
        role_name: web-server
        common_name: "{{ inventory_hostname }}.stackmaster.lab"
        ttl: 720h
      register: cert
      delegate_to: localhost
      no_log: true

    - name: Install certificate and key
      ansible.builtin.copy:
        content: "{{ item.content }}"
        dest: "{{ item.dest }}"
        mode: "{{ item.mode }}"
      loop:
        - { content: "{{ cert.data.data.certificate }}", dest:
/etc/nginx/tls/web.crt, mode: "0644" }
        - { content: "{{ cert.data.data.private_key }}", dest:
/etc/nginx/tls/web.key, mode: "0600" }
      no_log: true
```

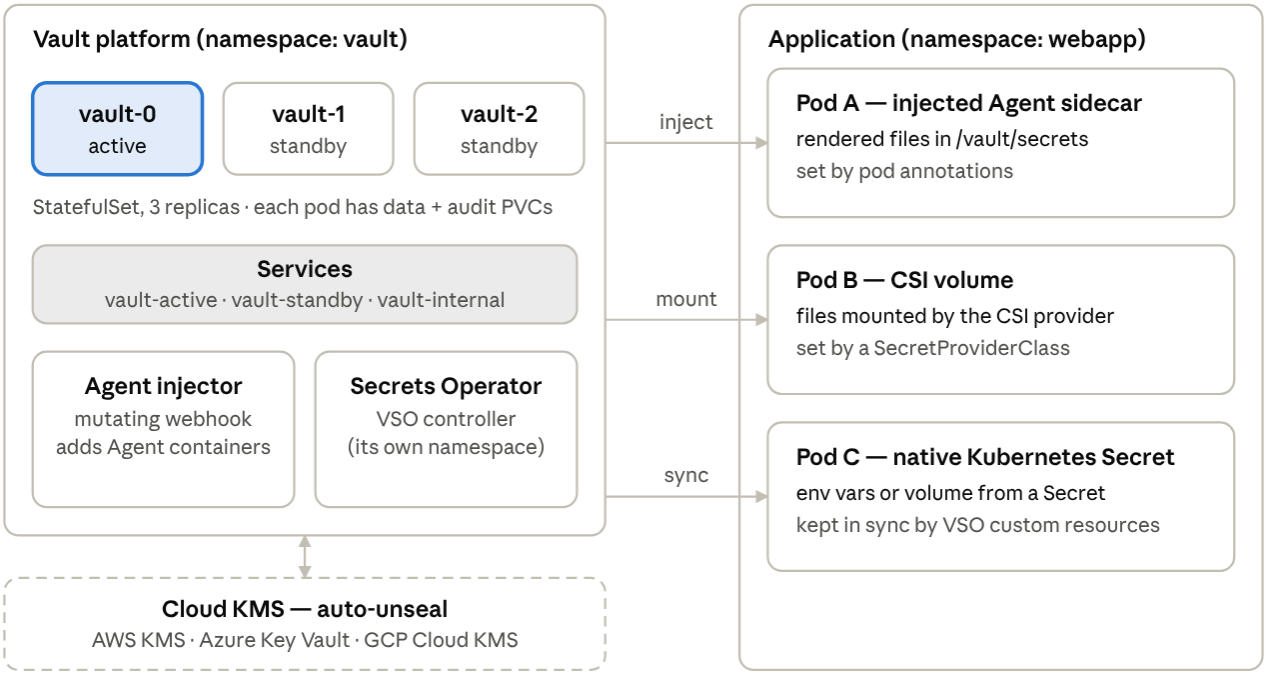
Other operations that suit playbooks: rolling upgrades (`serial: 1`, standbys first, then `vault operator step-down` on the leader), pruning old snapshots, revoking leases by prefix after an incident (`vault_write` on `sys/leases/revoke-prefix/...`), and — in labs only — Shamir unseal with keys held in Ansible Vault. Production clusters use auto-unseal (Module 3), so there is nothing to unseal by script.

> **Do Lab 3.8 — Terraform and Ansible automation** in the Day 3 Lab Guide: rebuild the KV, AppRole, Transit and PKI setup with Terraform, change a policy through `plan` / `apply`, then run the operational playbook (health check, snapshot, key rotation, KV write) and the certificate playbook.

Module 9 — Implementing Vault on Kubernetes

Day 2 Module 3 connected Kubernetes workloads to an external Vault with the Kubernetes auth method. This module goes further: it runs Vault itself inside the cluster as a production-style HA deployment, then compares the three ways pods receive secrets. It is written as an implementation document you can follow for a real cluster; Lab 3.9 runs the same steps on a local `kind` cluster.

One Vault cluster in Kubernetes, three ways for pods to get secrets



All three log in with Vault's Kubernetes auth method, using the pod's ServiceAccount token (Day 2 Module 3).

Vault on Kubernetes · HA Raft cluster and three delivery methods

9.1 Deployment choices

Decision	Recommended	Why
Where Vault runs	Dedicated namespace (ideally dedicated nodes via taints)	Limits who can exec into Vault pods or read their volumes
Install method	Official Helm chart `hashicorp/vault`	Maintained StatefulSet, services, PDB, injector, probes
Storage	Integrated Raft, 3 or 5 replicas, one per zone	No external Consul; quorum survives a zone loss
Unseal	Cloud KMS auto-unseal with workload identity (IRSA, Azure Workload Identity, GKE WI)	Pods restart often; nobody should type unseal keys
TLS	Certificates from cert-manager or Vault PKI, mounted as a Secret	The chart's default is plain HTTP
Secret delivery	VSO for most apps; Agent injector when files/templates or no K8s Secret are required	See 9.5

9.2 Prepare TLS and the namespace

The certificate must cover every name clients and peers use: `vault`, `vault.vault.svc`, `*.vault-internal`, `vault-active.vault.svc` and `127.0.0.1`.

```
kubectl create namespace vault

# vault.crt / vault.key / vault.ca from your CA (cert-manager, Vault PKI or
openssl)
kubectl -n vault create secret generic vault-ha-tls \
  --from-file=vault.crt=vault.crt --from-file=vault.key=vault.key --from-
file=vault.ca=vault.ca

# AWS example: KMS key ID for auto-unseal; the IAM role is bound to the
ServiceAccount (IRSA)
kubectl -n vault create secret generic vault-kms --from-
literal=AWS_KMS_KEY_ID=<kms-key-id>
```

9.3 Helm values for an HA Raft cluster

```
# vault-values.yaml
global:
  enabled: true
  tlsDisable: false

injector:
  enabled: true
  replicas: 2

server:
  image:
    repository: hashicorp/vault
    tag: "2.1.2"
  resources:
    requests: { memory: 256Mi, cpu: 250m }
    limits:   { memory: 1Gi }
  serviceAccount:
    annotations:
      eks.amazonaws.com/role-arn: arn:aws:iam::<account>:role/vault-unseal
  extraEnvironmentVars:
    VAULT_CACERT: /vault/userconfig/vault-ha-tls/vault.ca
  extraSecretEnvironmentVars:
    - envName: AWS_KMS_KEY_ID
      secretName: vault-kms
      secretKey: AWS_KMS_KEY_ID
  volumes:
    - name: userconfig-vault-ha-tls
      secret: { secretName: vault-ha-tls, defaultMode: 420 }
  volumeMounts:
    - name: userconfig-vault-ha-tls
      mountPath: /vault/userconfig/vault-ha-tls
      readOnly: true
  dataStorage:  { enabled: true, size: 10Gi, storageClass: gp3 }
  auditStorage: { enabled: true, size: 10Gi, storageClass: gp3 }
  ha:
    enabled: true
    replicas: 3
    raft:
      enabled: true
      setNodeId: true
      config: |
        ui = true
        cluster_name = "stackmaster-k8s"
        disable_mlock = true
```

```
        listener "tcp" {
          address         = "[::]:8200"
          cluster_address = "[::]:8201"
          tls_cert_file   = "/vault/userconfig/vault-ha-tls/vault.crt"
          tls_key_file    = "/vault/userconfig/vault-ha-tls/vault.key"
          tls_client_ca_file = "/vault/userconfig/vault-ha-tls/vault.ca"
          telemetry { unauthenticated_metrics_access = true }
        }

        storage "raft" {
          path = "/vault/data"
          retry_join {
            leader_api_addr     = "https://vault-0.vault-internal:8200"
            leader_ca_cert_file = "/vault/userconfig/vault-ha-tls/vault.ca"
          }
          retry_join {
            leader_api_addr     = "https://vault-1.vault-internal:8200"
            leader_ca_cert_file = "/vault/userconfig/vault-ha-tls/vault.ca"
          }
          retry_join {
            leader_api_addr     = "https://vault-2.vault-internal:8200"
            leader_ca_cert_file = "/vault/userconfig/vault-ha-tls/vault.ca"
          }
        }

        seal "awskms" {
          region = "ap-south-1"
          # kms_key_id is read from the AWS_KMS_KEY_ID environment variable
        }

        telemetry {
          prometheus_retention_time = "24h"
          disable_hostname          = true
        }

        service_registration "kubernetes" {}

ui:
  enabled: true
  serviceType: ClusterIP     # expose through an Ingress with TLS, never a
public LoadBalancer
```

The chart already sets pod anti-affinity (one Vault pod per node), a PodDisruptionBudget (`maxUnavailable: 1`) and the `OnDelete` update strategy, so Vault pods are only replaced when you delete them — which is how upgrades stay controlled.

9.4 Install, initialise and verify

```
helm repo add hashicorp https://helm.releases.hashicorp.com && helm repo
update
helm install vault hashicorp/vault -n vault -f vault-values.yaml
kubectl -n vault get pods            # vault-0..2 Running but 0/1 Ready: not
initialised yet

# Initialise once, on vault-0. With auto-unseal you get recovery keys, not
unseal keys.
kubectl -n vault exec vault-0 -- vault operator init -recovery-shares=5 -
recovery-threshold=3 -format=json > vault-init.json
# Store vault-init.json in your offline secret store, then delete it from the
workstation.

# vault-1 and vault-2 join through retry_join and unseal themselves via KMS
kubectl -n vault exec vault-0 -- sh -c 'VAULT_TOKEN=<root> vault operator
raft list-peers'
kubectl -n vault get pods -l app.kubernetes.io/name=vault -L vault-
active,vault-sealed
```

`service_registration "kubernetes"` adds the `vault-active`, `vault-sealed` and `vault-initialized` labels to each pod, which is how the `vault-active` Service always points at the leader.

Then configure the Kubernetes auth method from inside the cluster. When Vault runs in the same cluster it can use its own ServiceAccount to talk to the TokenReview API, so only the host is needed:

```
vault auth enable kubernetes
vault write auth/kubernetes/config
kubernetes_host="https://kubernetes.default.svc:443"

vault write auth/kubernetes/role/webapp \
  bound_service_account_names=webapp \
  bound_service_account_namespaces=webapp \
  audience=vault \
  token_policies=webapp token_ttl=1h
```

9.5 Three ways to deliver secrets to pods

	Agent injector	Secrets Store CSI provider	Vault Secrets Operator (VSO)
How	Webhook adds an init container and an Agent sidecar	CSI driver mounts a volume populated by the Vault provider	Controller syncs Vault data into native Kubernetes Secrets
App sees	Files in `/vault/secrets/`	Files in the CSI mount (optionally a synced Secret)	A normal Secret: env vars or volume
Templating	Full Agent templates (Module 4)	No (raw values)	Go templates in `spec.destination.transformation`
Renewal / rotation	Sidecar renews leases and re-renders	Re-read on rotation interval	Refreshes and can restart Deployments (`rolloutRestartTargets`)
Dynamic secrets, PKI	Yes	Yes (dynamic only at mount)	Yes (`VaultDynamicSecret`, `VaultPKISecret`)
Secret stored in etcd	No	No (unless sync enabled)	Yes — enable etcd encryption and tight RBAC
Extra per-pod cost	One sidecar per pod	None (DaemonSet)	None (one controller)
Choose when	Files and templates needed, no Secret allowed in etcd	You already use the CSI driver for other providers	Default for most apps; GitOps-friendly CRDs

Agent injector — annotations on the pod template:

```
spec:
  template:
    metadata:
      annotations:
        vault.hashicorp.com/agent-inject: "true"
        vault.hashicorp.com/role: "webapp"
        vault.hashicorp.com/agent-inject-secret-db.env: "secret/data/webapp"
        vault.hashicorp.com/agent-inject-template-db.env: |
          {{- with secret "secret/data/webapp" -}}
          DB_USER={{ .Data.data.db_user }}
          DB_PASSWORD={{ .Data.data.db_password }}
          {{- end }}
        vault.hashicorp.com/tls-secret: "vault-ha-tls"
        vault.hashicorp.com/ca-cert: "/vault/tls/vault.ca"
    spec:
      serviceAccountName: webapp
```

Vault Secrets Operator — install, then three custom resources:

```
helm install vault-secrets-operator hashicorp/vault-secrets-operator \
  -n vault-secrets-operator-system --create-namespace
```

```
apiVersion: secrets.hashicorp.com/v1beta1
kind: VaultConnection
metadata: { name: vault, namespace: webapp }
spec:
  address: https://vault.vault.svc:8200
  caCertSecretRef: vault-ca          # Secret in this namespace holding
ca.crt
---
apiVersion: secrets.hashicorp.com/v1beta1
kind: VaultAuth
metadata: { name: webapp, namespace: webapp }
spec:
  vaultConnectionRef: vault
  method: kubernetes
  mount: kubernetes
  kubernetes:
    role: webapp
    serviceAccount: webapp
    audiences: ["vault"]
---
apiVersion: secrets.hashicorp.com/v1beta1
kind: VaultStaticSecret
metadata: { name: webapp-db, namespace: webapp }
```

```
spec:
  vaultAuthRef: webapp
  mount: secret
  type: kv-v2
  path: webapp
  refreshAfter: 60s
  destination:
    name: webapp-db               # the Kubernetes Secret VSO creates and
owns
    create: true
  rolloutRestartTargets:
    - kind: Deployment
      name: webapp
```

The same pattern gives short-lived certificates with `VaultPKISecret` (`mount: pki_int`, `role: web-server`, `commonName`, `format: pem`, `expiryOffset: 1h`) and database credentials with `VaultDynamicSecret` (`mount: database`, `path: creds/webapp`).

9.6 Operating Vault on Kubernetes

- **Upgrades:** change `server.image.tag`, `helm upgrade`, then delete the standby pods one at a time (`kubectl delete pod vault-2`), wait for Ready and `raft autopilot state` healthy, and delete the active pod last — a standby takes over.
- **Snapshots:** a CronJob that logs in with its own Kubernetes auth role (policy: `read` on `sys/storage/raft/snapshot`) and uploads `vault operator raft snapshot save` output to object storage.
- **Monitoring:** scrape `/v1/sys/metrics?format=prometheus` with the Prometheus Operator (`serverTelemetry.serviceMonitor.enabled: true` in the chart) and alert on the Module 6 metrics.
- **Network policy:** allow 8200 from app namespaces, 8201 only between Vault pods, and egress to the KMS endpoint.
- **Audit:** the `auditStorage` PVC holds the file device; ship it with a sidecar or node agent, and add a `socket` device as the second sink.

Production checklist for Kubernetes

- [ ] Dedicated namespace and nodes; RBAC denies `exec` into Vault pods for non-operators
- [ ] TLS on every listener; the CA distributed to clients and to VSO/injector
- [ ] Auto-unseal through workload identity — no static cloud keys in Secrets
- [ ] 3 or 5 replicas across zones; PDB and anti-affinity in place
- [ ] Storage class with `reclaimPolicy: Retain` for data PVCs

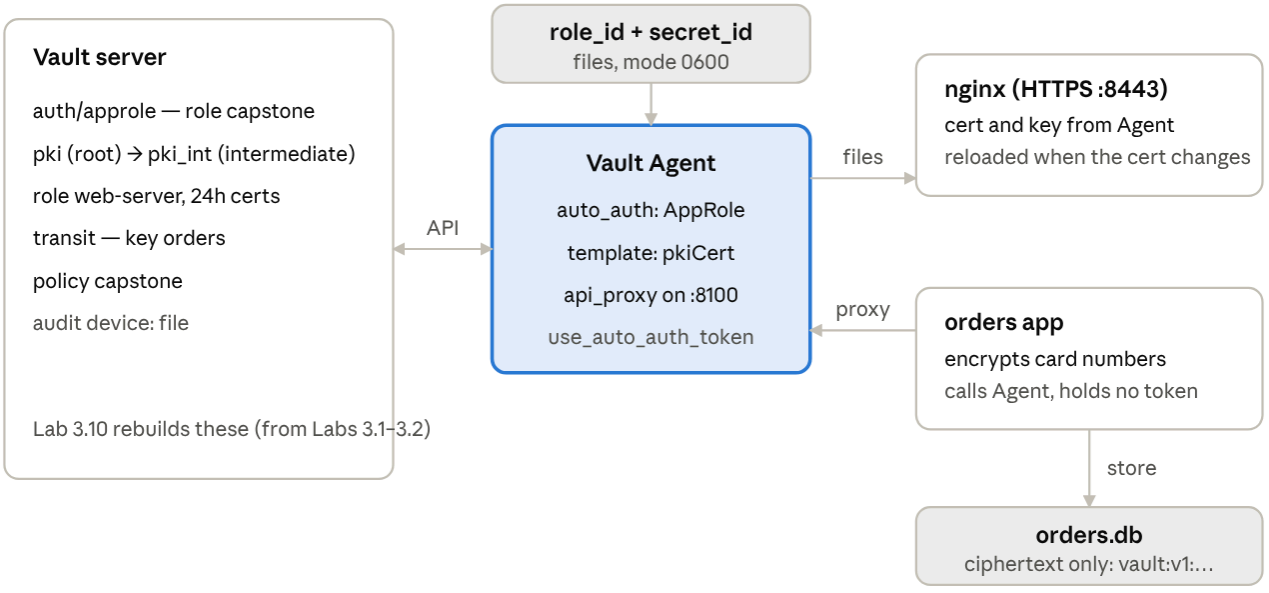
- [ ] etcd encryption at rest enabled if VSO or CSI sync writes Kubernetes Secrets
- [ ] Automated snapshots copied off-cluster, and a restore rehearsed
- [ ] Root token revoked after setup; recovery keys split among custodians

> **Do Lab 3.9 — Vault HA on Kubernetes with Helm and VSO** in the Day 3 Lab Guide: create a `kind` cluster, install the chart with three Raft replicas, initialise and join, configure Kubernetes auth, inject a secret with Agent annotations, then sync the same secret with VSO and watch a rotation restart the Deployment.

Module 10 — Capstone: PKI + Transit + Vault Agent end to end

The capstone joins the three Day 3 building blocks into one small but realistic service, **StackMaster Orders**. A web front end serves HTTPS with a short-lived certificate from Vault PKI; an orders application encrypts card numbers with Transit before they reach the database; and Vault Agent handles every interaction with Vault, so neither nginx nor the application ever holds a Vault token.

The app never holds a Vault token: Agent logs in, renews the cert and proxies Transit calls



capstone architecture · PKI + Transit + Vault Agent

10.1 What each part proves

Part	Built in	Proves
Two-tier PKI, role `web-server` (24 h certificates)	Module 1 / Lab 3.1	Certificates are issued on demand and expire quickly — no long-lived certs to leak
Transit key `orders`	Module 2 / Lab 3.2	The database holds only `vault:v1:…` ciphertext; the key never leaves Vault
AppRole role `capstone` + policy `capstone`	Day 2 Modules 2–3	The machine identity gets exactly the paths it needs, nothing more
Vault Agent: auto-auth, `pkiCert` template with `exec`, API proxy	Module 4 / Lab 3.4	Login, renewal and reloads happen without code in the app
Audit device	Module 6 / Lab 3.6	Every issue, encrypt and decrypt is traceable to the `capstone` role

10.2 The policy

```
# capstone.hcl
path "pki_int/issue/web-server" {
  capabilities = ["update"]
}
path "transit/encrypt/orders" {
  capabilities = ["update"]
}
path "transit/decrypt/orders" {
  capabilities = ["update"]
}
path "transit/rewrap/orders" {          # lets the app upgrade old ciphertext
after a rotation
  capabilities = ["update"]
}
# no read on transit/keys/*, no sys/*, no secret/*
```

10.3 The Agent configuration

```
# agent-capstone.hcl
vault {
  address = "https://127.0.0.1:8200"
  ca_cert = "/opt/capstone/tls/vault-ca.pem"
```

```
}

auto_auth {
  method "approle" {
    config = {
      role_id_file_path                   = "/opt/capstone/role_id"
      secret_id_file_path                 = "/opt/capstone/secret_id"
      remove_secret_id_file_after_reading = false   # lab only; true in
production
    }
  }
  sink "file" {
    config = { path = "/opt/capstone/agent-token", mode = 0600 }
  }
}

# The app talks to Agent; Agent adds its own token to each request
cache {}
api_proxy {
  use_auto_auth_token = "force"
}
listener "tcp" {
  address     = "127.0.0.1:8100"
  tls_disable = true                # loopback only
}

# One template renders the cert, key and CA chain into a single bundle file
template {
  destination = "/opt/capstone/tls/web.pem"
  perms       = "0600"
  contents    = <<EOT
{{- with pkiCert "pki_int/issue/web-server"
"common_name=orders.stackmaster.lab" "ttl=24h" -}}
{{ .Cert }}{{ .CA }}{{ .Key }}
{{ .Key  | writeToFile "/opt/capstone/tls/web.key" "" "" "0600" }}
{{ print .Cert "\n" .CA | writeToFile "/opt/capstone/tls/web.crt" "" ""
"0644" }}
{{ .CA   | writeToFile "/opt/capstone/tls/ca.crt" "" "" "0644" }}
{{- end -}}
EOT
  exec {
    command = ["sh", "-c", "nginx -s reload || true"]
  }
}
```

`pkiCert` reads the destination file first: if it already holds a certificate that is not close to expiry, Agent reuses it instead of issuing a new one on every restart. That is why the destination must contain the certificate itself.

10.4 The application side

The orders app needs only an HTTP client and the Agent address — no token, no Vault SDK:

```
# orders.py (extract)
import base64, requests
AGENT = "http://127.0.0.1:8100"

def encrypt(card: str) -> str:
    b64 = base64.b64encode(card.encode()).decode()
    r = requests.post(f"{AGENT}/v1/transit/encrypt/orders", json=
{"plaintext": b64}, timeout=5)
    r.raise_for_status()
    return r.json()["data"]["ciphertext"]          # vault:v1:...

def decrypt(ct: str) -> str:
    r = requests.post(f"{AGENT}/v1/transit/decrypt/orders", json=
{"ciphertext": ct}, timeout=5)
    r.raise_for_status()
    return base64.b64decode(r.json()["data"]["plaintext"]).decode()
```

10.5 Success criteria

- [ ] `curl --cacert root_ca.pem https://orders.stackmaster.lab:8443` returns the page, and `openssl s_client` shows a certificate issued by the StackMaster intermediate CA with a 24-hour lifetime.
- [ ] A new order stored in `orders.db` holds `vault:v1:…`, never the card number.
- [ ] Rotating the `orders` key makes new rows `vault:v2:…`; old rows still decrypt; `transit/rewrap` upgrades them.
- [ ] Restarting Agent reuses the still-valid certificate cached in its destination file: no new pki_int/issue request appears in the audit log.
- [ ] Forcing a short cert TTL (for example `ttl=3m`) shows Agent renewing the certificate and reloading nginx without a restart.
- [ ] The audit log shows each `pki_int/issue/web-server` and `transit/encrypt/orders` request with display name `approle`.

10.6 Stretch goals

- Run the same design on Kubernetes (Module 9): Agent injector annotations for the cert, VSO `VaultPKISecret` for the Ingress TLS Secret.
- Express the whole Vault side (mounts, roles, policy, AppRole) in Terraform (Module 8) and rebuild it from scratch.
- Move the Vault server to Transit auto-unseal (Lab 3.3) and confirm the capstone survives a Vault restart with no human action.

> **Do Lab 3.10 — Capstone** in the Day 3 Lab Guide: it builds this design step by step, then runs each success check above.

Day 3 cheat sheet

Task	Command
Enable and tune PKI	`vault secrets enable pki` · `vault secrets tune -max-lease-ttl=87600h pki`
Root CA	`vault write pki/root/generate/internal common_name="StackMaster Root CA" ttl=87600h issuer_name=root-2026`
Intermediate CSR → sign → import	`vault write -format=json pki_int/intermediate/generate/internal common_name=...` → `vault write pki/root/sign-intermediate csr=@pki_int.csr` → `vault write pki_int/intermediate/set-signed certificate=@int.pem`
Role and issue	`vault write pki_int/roles/web-server allowed_domains=stackmaster.lab allow_subdomains=true max_ttl=720h` · `vault write pki_int/issue/web-server common_name=www.stackmaster.lab ttl=24h`
Revoke and tidy	`vault write pki_int/revoke serial_number=<sn>` · `vault write pki_int/tidy tidy_cert_store=true tidy_revoked_certs=true`
PKI health	`vault pki health-check pki_int`

Task	Command
Transit key	`vault secrets enable transit` · `vault write -f transit/keys/orders`
Encrypt / decrypt	`vault write transit/encrypt/orders plaintext=$(base64 <<< "4111...")` · `vault write transit/decrypt/orders ciphertext=vault:v1:...`
Rotate / rewrap / floor	`vault write -f transit/keys/orders/rotate` · `vault write transit/rewrap/orders ciphertext=...` · `vault write transit/keys/orders/config min_decryption_version=2`
Seal migration	Add a `seal` stanza, restart, then `vault operator unseal -migrate` with the old keys
Run Agent	`vault agent -config=agent.hcl`
Raft peers and health	`vault operator raft list-peers` · `vault operator raft autopilot state`
Join / remove / step down	`vault operator raft join https://vault-1:8200` · `vault operator raft remove-peer vault-3` · `vault operator step-down`
Snapshots	`vault operator raft snapshot save backup.snap` · `vault operator raft snapshot restore backup.snap`
Health and metrics	`curl $VAULT_ADDR/v1/sys/health` · `vault read sys/metrics`
Audit devices	`vault audit enable file file_path=/var/log/vault/vault-audit.log` · `vault audit list -detailed` · `vault write sys/audit-hash/file input=...`
Troubleshoot	`journalctl -u vault -f` · `vault monitor -log-level=debug` · `vault token capabilities <path>` · `vault debug -duration=2m`
Terraform	`terraform init` · `terraform plan` · `terraform apply` (provider `hashicorp/vault`)

Task	Command
Ansible	`ansible-galaxy collection install community.hashi_vault` · `ansible-playbook vault-ops.yml`
Kubernetes	`helm install vault hashicorp/vault -n vault -f vault-values.yaml` · `kubectl -n vault exec vault-0 -- vault operator init`

Day 3 knowledge check

1. Why is the root CA usually kept offline (or in a separate mount) while an intermediate CA issues leaf certificates?
2. What is the difference between `pki_int/issue/<role>` and `pki_int/sign/<role>`, and when would you use `sign`?
3. A Transit key has been rotated to version 4 and `min_decryption_version` is set to 3. What happens when an app tries to decrypt `vault:v2:…`, and how should the data have been prepared?
4. Name two ways Transit auto-unseal differs from cloud KMS auto-unseal, and one risk it introduces.
5. After migrating to auto-unseal, which keys does `vault operator generate-root` need: the old unseal keys or the recovery keys?
6. In Vault Agent, what is the difference between `template`, `env_template` + `exec`, and `api_proxy`? Give one use case for each.
7. A three-node Raft cluster loses two nodes. Why can't it elect a leader, and what are the two documented ways to recover?
8. What is the difference between a performance replica and a DR replica (Enterprise), and which one can serve client reads?
9. Vault suddenly refuses every request even though it is unsealed, and the server log shows audit write errors. What happened and how do you prevent it?
10. Which two `sys/health` status codes does a healthy standby return with and without `standbyok=true`?
11. Why should Terraform configure Vault but not deliver application secrets, and how do you protect the Terraform state?
12. You must give 40 Kubernetes Deployments a database password, and the security team forbids Kubernetes Secrets in etcd. Which delivery method do you choose, and why?

Answers

1. Compromise of the issuing CA can be contained by revoking and replacing the intermediate; a compromised root means rebuilding trust everywhere. Keeping the root rarely used limits its exposure.
2. `issue` makes Vault generate the private key and return it; `sign` signs a CSR the client created, so the private key never leaves the client (for example, an HSM-backed server or a device).
3. Decryption fails because version 2 is below the minimum. Before raising the floor, all stored ciphertext should have been rewrapped with `transit/rewrap` to the latest version.
4. Transit seal depends on another Vault cluster (not a cloud service) and works on-prem or air-gapped; the token and the unsealing cluster become a dependency — if the unsealing Vault is down, the protected cluster cannot unseal after a restart.
5. The recovery keys. With auto-unseal, the unseal (root) key is protected by the KMS and the recovery keys authorise operations such as generate-root and rekey.
6. `template` renders files (config with a DB password, a cert via `pkiCert`); `env_template` + `exec` starts the app as a child process with secrets in environment variables and restarts it when they change; `api_proxy` lets an app call Vault through Agent, which adds the auto-auth token (Transit calls in the capstone).
7. Raft needs a majority (2 of 3) to elect a leader. Recover with `peers.json` on the surviving node to form a one-node cluster and re-join others, or build a new cluster and restore the latest snapshot.
8. A performance replica serves reads (and forwards writes) with its own tokens and leases; a DR replica holds a full copy including tokens and leases but serves no client traffic until promoted.
9. Vault blocks requests when it cannot write to any enabled audit device (for example a full disk). Run at least two audit devices, monitor `vault_audit_log_request_failure`, and rotate logs with SIGHUP.
10. Without the parameter a standby returns 429; with `standbyok=true` it returns 200.
11. Values Terraform handles end up in its state, and app secrets should be fetched and renewed at run time. Use an encrypted remote backend with restricted access, and write-only arguments where the provider supports them.
12. The Agent injector (or the CSI provider without sync): secrets go straight into the pod as files and never become Kubernetes Secrets in etcd. VSO would need Kubernetes Secrets.

Coming next: this completes the three-day programme. For revision, re-run the capstone (Lab 3.10) from a clean Vault server, and use the Day 1–3 cheat sheets as the reference card.

Sources

- Vault documentation — PKI secrets engine
- Vault documentation — Transit secrets engine
- Vault documentation — Seal configuration and auto-unseal
- Vault Agent templates
- Integrated storage (Raft) and autopilot
- Telemetry metrics reference
- Audit devices
- Production hardening
- Vault Helm chart configuration
- Vault Secrets Operator
- Terraform Vault provider
- community.hashi_vault Ansible collection
- Vault release notes